

SMART BUFFET MEAL PLANNER USING ALGORITHMS

1. Abstract

The **Smart Buffet Meal Planner** is an optimization-based system designed to help users select the best combination of meals under a given budget while maximizing nutritional value. The system models the problem as a **0/1 Knapsack optimization problem**, where each meal combo represents an item with cost and nutritional attributes.

The project implements and compares:

- **Greedy Algorithm** – fast, heuristic-based selection
- **Dynamic Programming** – optimal solution using exhaustive subproblem evaluation

Additionally, the system integrates:

- **Binary Search** for efficient retrieval of meal combinations
- **Weekly Buffet Planning** for real-world usability

The system demonstrates how classical algorithms can be applied to practical domains such as **canteen management, diet planning, and resource optimization**.

2. Introduction

Meal planning in real-life scenarios (college mess, hostels, canteens) involves balancing:

- Budget constraints
- Nutritional requirements
- Availability of food options

Manual selection is inefficient and often suboptimal. This project addresses the issue by applying **algorithmic decision-making techniques**.

The uniqueness of this project lies in:

- Using **meal combos (buffet style)** instead of individual items
- Comparing **two algorithmic paradigms**
- Providing **automated weekly planning**

3. Problem Statement

Given:

- A set of meal combinations with attributes:
 - Cost
 - Protein
 - Calories
- A fixed user budget

Objective:

Select a subset of meal combinations such that:

- Total cost $\leq$ Budget
- Total nutrition score is **maximized**

Constraints:

- Each combo can be selected at most once
- User preferences (veg/non-veg, meal type) must be respected

4. System Objectives

- Design an efficient meal optimization system
- Compare Greedy vs Dynamic Programming performance
- Simulate real-world buffet meal selection
- Enable fast search using Binary Search
- Generate a weekly meal plan automatically

5. Literature / Concept Background

5.1 Knapsack Problem

The problem is a variation of the 0/1 Knapsack Problem, where:

- Items = Meal combos
- Weight = Cost
- Value = Nutrition score

5.2 Greedy Algorithm

- Selects items based on local optimal choice
- Uses value-to-cost ratio

Limitation:

- Does not guarantee global optimal solution

5.3 Dynamic Programming

- Breaks problem into subproblems
- Stores results in a table
- Guarantees optimal solution

5.4 Binary Search

- Efficient searching technique
- Works on sorted data
- Reduces search complexity significantly

6. Methodology

6.1 Data Representation

Each meal combo is defined as:

```
struct MealCombo {  
    name  
  
    cost  
  
    protein  
  
    calories  
  
    veg  
  
    category  
  
}
```

6.2 Nutrition Score Formula

$\text{Score} = (2 \times \text{Protein}) + (0.1 \times \text{Calories})$

Reasoning:

- Protein is given higher weight (important nutrient)
- Calories contribute moderately

6.3 Workflow

User Input → Filter Meals → Apply Algorithms → Compare Results → Generate Weekly Plan → Search Feature

7. Algorithms Implementation

♦ 7.1 Greedy Algorithm

Steps:

1. Calculate value-to-cost ratio
2. Sort meals in descending order
3. Select items until budget is exhausted

Pseudo Code:

Sort items by (value / cost) descending

Initialize total_cost = 0

for each item:

 if total_cost + item.cost ≤ budget:

 select item

 update total_cost

Time Complexity:

$O(n^2)$

Space Complexity:

$O(n)$

♦ 7.2 Dynamic Programming (0/1 Knapsack)

DP Table Definition:

$dp[i][w]$ = maximum value using first i items and budget w

Transition:

$dp[i][w] = \max(dp[i-1][w], value_i + dp[i-1][w - cost_i])$

Transition:

```
if cost[i] ≤ w:
```

```
    dp[i][w] = max(  
        dp[i-1][w],  
        value[i] + dp[i-1][w - cost[i]]  
    )
```

```
else:
```

```
    dp[i][w] = dp[i-1][w]
```

Time Complexity:

$O(n \times \text{budget})$

Space Complexity:

$O(n \times \text{budget})$

♦ 7.3 Binary Search

Used for searching meal combos by name.

Steps:

1. Sort data alphabetically
2. Apply binary search

Pseudo Code:

```
low = 0, high = n-1
```

```
while low ≤ high:
```

```
    mid = (low + high) / 2
```

```

if target == arr[mid]:
    return mid

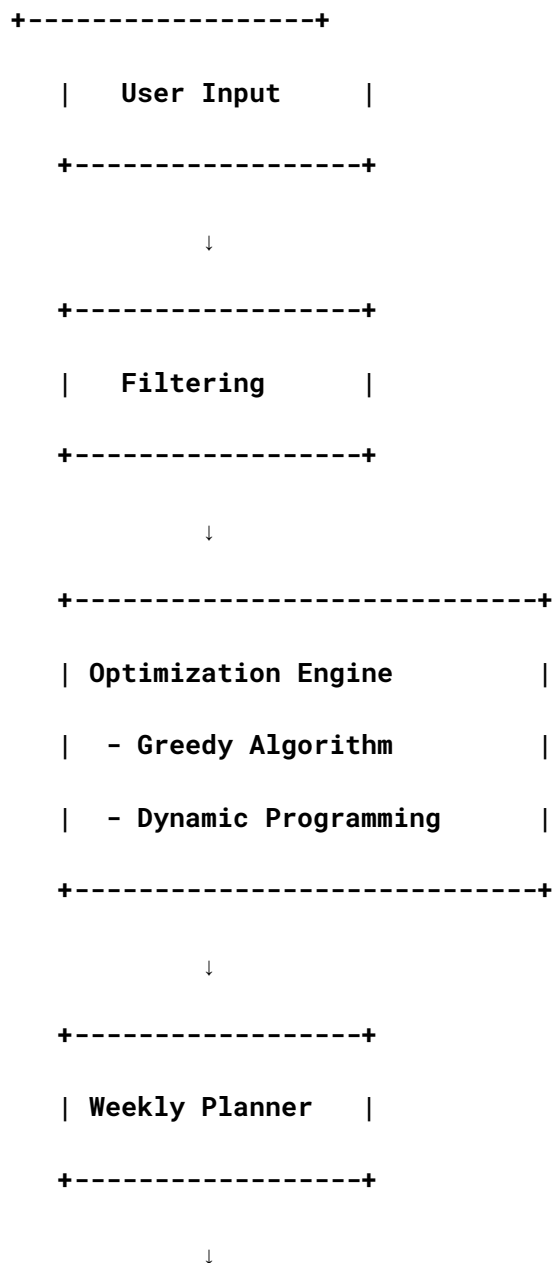
else if target > arr[mid]:
    low = mid + 1

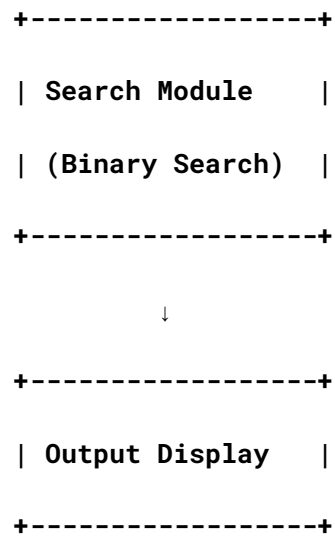
else:
    high = mid - 1

```

Time Complexity:

$O(\log n)$





CODE

```
#include <stdio.h>

#include <string.h>

#define MAX 100

#define MAX_BUDGET 1000

// ----- STRUCT -----

typedef struct {

    char name[60];

    int cost;

    int protein;

    int calories;

    int veg;

    char category[20];

} MealCombo;
```

```
// ----- DATASET -----

MealCombo combos[] = {

    // BREAKFAST

    {"Idli + Sambar + Chutney", 50, 6, 180, 1, "breakfast"},

    {"Dosa + Chutney + Milk", 60, 8, 250, 1, "breakfast"},

    {"Oats + Milk + Fruits", 55, 9, 220, 1, "breakfast"},

    {"Bread + Butter + Milk", 45, 6, 210, 1, "breakfast"},

    {"Eggs + Toast + Juice", 70, 12, 260, 0, "breakfast"},

    {"Poha + Tea", 40, 4, 180, 1, "breakfast"},

    {"Boiled Eggs + Milk", 60, 12, 180, 0, "breakfast"},

    {"Egg Sandwich + Tea", 70, 10, 220, 0, "breakfast"},

    {"Omelette + Bread", 65, 11, 210, 0, "breakfast"},

    // LUNCH

    {"Rice + Dal + Curd", 80, 12, 350, 1, "lunch"},

    {"Roti + Paneer Curry", 110, 18, 400, 1, "lunch"},

    {"Veg Biryani + Raita", 120, 10, 450, 1, "lunch"},

    {"Fried Rice + Manchurian", 130, 12, 500, 1, "lunch"},

    {"Chicken Biryani + Salad", 150, 22, 550, 0, "lunch"},

    {"Egg Curry + 2 Roti", 90, 14, 320, 0, "lunch"},

    {"Chicken Curry (Small) + Roti", 95, 16, 350, 0, "lunch"},

    {"Egg Fried Rice (Half)", 85, 12, 300, 0, "lunch"},

    // DINNER

    {"Roti + Dal Tadka + Veg", 100, 15, 380, 1, "dinner"},
```



```

    {"Paneer + Roti + Salad", 120, 20, 420, 1, "dinner"},

    {"Grilled Chicken + Rice", 160, 30, 500, 0, "dinner"},

    {"Fish Curry + Rice", 150, 25, 480, 0, "dinner"},

    {"Soy Chunk Curry + Roti", 110, 22, 400, 1, "dinner"},

    {"Egg Curry + Rice", 120, 18, 450, 0, "dinner"},

    {"Egg Curry + Rice (Small)", 95, 15, 340, 0, "dinner"},

    {"Chicken Soup + Bread", 80, 10, 200, 0, "dinner"},

    {"Egg Bhurji + Roti", 90, 13, 310, 0, "dinner"}

};

int n = sizeof(combos) / sizeof(combos[0]);

// ----- VALUE FUNCTION -----

float getValue(MealCombo item) {

    return item.protein * 2 + item.calories * 0.1;

}

// ----- FILTER -----

int filterItems(MealCombo filtered[], int veg_only, char meal_type[]) {

    int count = 0;

    for (int i = 0; i < n; i++) {

        if (veg_only && combos[i].veg == 0)

            continue;

        if (strcmp(meal_type, "all") != 0 &&

```

```

        strcmp(combos[i].category, meal_type) != 0)

        continue;

    filtered[count++] = combos[i];
}

return count;
}

// ----- DISPLAY -----

void displayItems(MealCombo items[], int count) {

    int totalCost = 0, totalProtein = 0, totalCalories = 0;

    float totalValue = 0;

    for (int i = 0; i < count; i++) {

        printf("• %s (₹%d, Protein:%dg, Calories:%d)\n",

            items[i].name,

            items[i].cost,

            items[i].protein,

            items[i].calories);

        totalCost += items[i].cost;

        totalProtein += items[i].protein;

        totalCalories += items[i].calories;

        totalValue += getValue(items[i]);

    }
}

```

```

printf("\nTotal Cost: ₹%d", totalCost);

printf("\nTotal Protein: %dg", totalProtein);

printf("\nTotal Calories: %d", totalCalories);

printf("\nNutrition Score: %.2f\n", totalValue);
}

// ----- SORT FOR BINARY SEARCH -----

void sortByName(MealCombo arr[], int count) {

    for (int i = 0; i < count - 1; i++) {

        for (int j = 0; j < count - i - 1; j++) {

            if (strcmp(arr[j].name, arr[j + 1].name) > 0) {

                MealCombo temp = arr[j];

                arr[j] = arr[j + 1];

                arr[j + 1] = temp;

            }

        }

    }

}

// ----- BINARY SEARCH -----

int binarySearch(MealCombo arr[], int count, char target[]) {

    int low = 0, high = count - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

```

```

        int cmp = strcmp(arr[mid].name, target);

        if (cmp == 0)

            return mid;

        else if (cmp < 0)

            low = mid + 1;

        else

            high = mid - 1;

    }

    return -1;
}

// ----- GREEDY -----

void greedy(MealCombo items[], int count, int budget) {

    for (int i = 0; i < count - 1; i++) {

        for (int j = 0; j < count - i - 1; j++) {

            float r1 = getValue(items[j]) / items[j].cost;

            float r2 = getValue(items[j + 1]) / items[j + 1].cost;

            if (r1 < r2) {

                MealCombo temp = items[j];

                items[j] = items[j + 1];

                items[j + 1] = temp;

            }

        }

    }
}

```

```

    }

    MealCombo selected[MAX];

    int selCount = 0, totalCost = 0;

    for (int i = 0; i < count; i++) {

        if (totalCost + items[i].cost <= budget) {

            selected[selCount++] = items[i];

            totalCost += items[i].cost;

        }

    }

    printf("\n--- GREEDY APPROACH ---\n");

    displayItems(selected, selCount);

}

// ----- DP -----

void dp(MealCombo items[], int count, int budget, MealCombo selected[], int *selCount) {

    float dp[MAX][MAX_BUDGET] = {0};

    for (int i = 1; i <= count; i++) {

        for (int w = 0; w <= budget; w++) {

            if (items[i - 1].cost <= w) {

                float include = getValue(items[i - 1]) +

                    dp[i - 1][w - items[i - 1].cost];
            }
        }
    }
}

```

```

        float exclude = dp[i - 1][w];

        dp[i][w] = (include > exclude) ? include : exclude;

    } else {

        dp[i][w] = dp[i - 1][w];

    }

}

}

int w = budget;

*selCount = 0;

for (int i = count; i > 0; i--) {

    if (dp[i][w] != dp[i - 1][w]) {

        selected[(*selCount)++] = items[i - 1];

        w -= items[i - 1].cost;

    }

}

printf("\n--- DYNAMIC PROGRAMMING (OPTIMAL) ---\n");

displayItems(selected, *selCount);

printf("\nOptimal Score: %.2f\n", dp[count][budget]);
}

// ----- WEEKLY PLAN -----

void weeklyBuffet(MealCombo items[], int count) {

```

```

char *days[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

printf("\n🍽️ WEEKLY BUFFET PLAN:\n\n");

for (int d = 0; d < 7; d++) {

    printf("%s:\n", days[d]);

    int b=-1, l=-1, din=-1;

    for (int i = 0; i < count; i++) {

        int idx = (i + d) % count;

        if (b == -1 && strcmp(items[idx].category, "breakfast") == 0)

            b = idx;

        else if (l == -1 && strcmp(items[idx].category, "lunch") == 0)

            l = idx;

        else if (din == -1 && strcmp(items[idx].category, "dinner") == 0)

            din = idx;

    }

    if (b != -1) printf(" 🍳 %s\n", items[b].name);

    if (l != -1) printf(" 🍲 %s\n", items[l].name);

    if (din != -1) printf(" 🍽️ %s\n", items[din].name);

    printf("\n");

}

```

```

}

// ----- MAIN -----

int main() {

    int budget, veg_choice;

    char meal_type[20];

    printf("🍽️ SMART BUFFET MEAL PLANNER\n");

    printf("Enter budget (₹): ");

    scanf("%d", &budget);

    if (budget <= 0 || budget > MAX_BUDGET) {

        printf("Invalid budget ❌\n");

        return 0;

    }

    printf("Veg only? (1=Yes, 0=No): ");

    scanf("%d", &veg_choice);

    printf("Meal type (breakfast/lunch/dinner/all): ");

    scanf("%s", meal_type);

    MealCombo filtered[MAX];

    int count = filterItems(filtered, veg_choice, meal_type);

```



```

if (count == 0) {

    printf("No combos available ✖\n");

    return 0;

}

// Greedy

MealCombo greedy_items[MAX];

for (int i = 0; i < count; i++)

    greedy_items[i] = filtered[i];

greedy(greedy_items, count, budget);

// DP

MealCombo dp_selected[MAX];

int dp_count = 0;

dp(filtered, count, budget, dp_selected, &dp_count);

// Weekly Plan

weeklyBuffet(dp_selected, dp_count);

// ----- SEARCH -----

char searchName[60];

printf("\n🔍 Enter combo name to search: ");

scanf("%[^\n]", searchName);

```

```
sortByName(filtered, count);

int index = binarySearch(filtered, count, searchName);

if (index != -1) {

    printf("\n✅ Found Combo:\n");

    printf("%s (₹%d, Protein:%dg, Calories:%d)\n",

           filtered[index].name,

           filtered[index].cost,

           filtered[index].protein,

           filtered[index].calories);

} else {

    printf("❌ Combo not found\n");

}

return 0;

}
```

9. Sample Output

GREEDY APPROACH

- Poha + Tea (₹40)
- Eggs + Toast + Juice (₹70)

DYNAMIC PROGRAMMING

- Egg Curry + 2 Roti (₹90)
- Oats + Milk + Fruits (₹55)

Total Cost: ₹145

Nutrition Score: 78.5

WEEKLY BUFFET PLAN:

Mon:

Breakfast: Oats + Milk + Fruits

Lunch: Egg Curry + 2 Roti

Dinner: Egg Curry + Rice

```
🍽️ SMART BUFFET MEAL PLANNER
Enter budget (₹): 1000
Veg only? (1=Yes, 0=No): 0
Meal type (breakfast/lunch/dinner/all): dinner

--- GREEDY APPROACH ---
• Soy Chunk Curry + Roti (₹110, Protein:22g, Calories:400)
• Grilled Chicken + Rice (₹160, Protein:30g, Calories:500)
• Paneer + Roti + Salad (₹120, Protein:20g, Calories:420)
• Roti + Dal Tadka + Veg (₹100, Protein:15g, Calories:380)
• Egg Curry + Rice (₹120, Protein:18g, Calories:450)
• Egg Curry + Rice (Small) (₹95, Protein:15g, Calories:340)
• Fish Curry + Rice (₹150, Protein:25g, Calories:480)
• Egg Bhurji + Roti (₹90, Protein:13g, Calories:310)

Total Cost: ₹945
Total Protein: 158g
Total Calories: 3280
Nutrition Score: 644.00

--- DYNAMIC PROGRAMMING (OPTIMAL) ---
• Egg Bhurji + Roti (₹90, Protein:13g, Calories:310)
• Egg Curry + Rice (Small) (₹95, Protein:15g, Calories:340)
• Egg Curry + Rice (₹120, Protein:18g, Calories:450)
• Soy Chunk Curry + Roti (₹110, Protein:22g, Calories:400)
• Fish Curry + Rice (₹150, Protein:25g, Calories:480)
• Grilled Chicken + Rice (₹160, Protein:30g, Calories:500)
• Paneer + Roti + Salad (₹120, Protein:20g, Calories:420)
• Roti + Dal Tadka + Veg (₹100, Protein:15g, Calories:380)
```

Total Cost: ₹945
Total Protein: 158g
Total Calories: 3280
Nutrition Score: 644.00

Optimal Score: 644.00

 WEEKLY BUFFET PLAN:

Mon:

● Egg Bhurji + Roti

Tue:

● Egg Curry + Rice (Small)

Thu:

● Soy Chunk Curry + Roti

Fri:

● Fish Curry + Rice

Sat:

● Grilled Chicken + Rice

Sun:

● Paneer + Roti + Salad

 Enter combo name to search: Grilled Chicken + Rice

 Found Combo:

Grilled Chicken + Rice (₹160, Protein:30g, Calories:500)

10. Comparative Analysis

Speed	Fast	Slower
Accuracy	Approximate	Optimal
Complexity	$O(n^2)$	$O(n \times \text{budget})$
Use Case	Quick choice	Best solution

11. Advantages

- Real-world applicability
- Efficient optimization
- Algorithm comparison
- Scalable system
- Easy to extend

12. Limitations

- Static dataset (no real-time updates)
- Simplified nutrition formula
- No user personalization

13. Future Enhancements

-  Web-based UI (React + Node.js)
-  Graph comparison (Greedy vs DP)
-  AI-based recommendations
-  Diet-specific plans (keto, high protein)
-  Database integration

14. Applications

- College mess systems
- Food delivery platforms
- Health and fitness apps
- Budget optimization tools

15. Conclusion

This project successfully demonstrates the application of **algorithmic techniques in real-world optimization problems**.

By combining:

- Greedy (efficiency)
- Dynamic Programming (optimality)
- Binary Search (speed)

The system achieves a balance between **performance, accuracy, and usability**.